C3ache: Towards Hierarchical Cache-Centric Computing for Sparse Matrix Multiplication on GPGPUs

Xiaojie Li
Sun Yat-sen University
Guangzhou, Guangdong, China
lixj263@mail2.sysu.edu.cn

Mingyu Wang*
Sun Yat-Sen University
Guangzhou, Guangdong, China
wangmingyu@mail.sysu.edu.cn

Baiqing Zhong
Sun Yat-Sen University
Guangzhou, Guangdong, China
zhongbq@mail2.sysu.edu.cn

Haiqiu Huang
Sun Yat-Sen University
Guangzhou, Guangdong, China
huanghq73@mail2.sysu.edu.cn

Guangjie Cao
Sun Yat-sen University
Guangzhou, Guangdong, China
caogj@mail2.sysu.edu.cn

Zhiyi Yu
Sun Yat-sen University
Guangzhou, Guangdong, China
yuzhiyi@mail.sysu.edu.cn

Abstract

Sparse matrix multiplications (SPMMs) are fundamental kernels in various domains and are highly demanded to be executed on general-purpose graphics processing units (GPGPUs). However, it is a challenge to efficiently execute SPMMs across varying sparsity regimes on GPGPUs due to the significant variations in sparsity patterns across different domains. Most of the state-of-the-art works enhance GPGPUs by integrating dedicated accelerator units in processor, which prioritize computational density over memory subsystem optimizations. As sparsity increases, the overhead from expensive memory accesses and redundant input fetches in sparse matrices constrains the effectiveness of processor-centric optimization schemes. Consequently, these optimizations become suboptimal for workloads dominated by irregular memory access patterns and low arithmetic intensity.

To address these problems, we proposed a hierarchical cache-centric computing architecture with hybrid dataflow, C^3ache, to achieve near-optimal memory efficiency and data reuse. First, the hybrid dataflow based on the outer-product dataflow and Gustavson's dataflow is proposed to decouple the SPMM computation into two distinct phases (multiplication and merging) with different behavioral characteristics and align with the memory access pattern. Furthermore, C^3ache restructures the cache hierarchy through in-cache computing, transforming the two levels of cache into large-scale data parallel processing in memory (PIM) units and in-situ merging PIM units respectively to map the two computation phases of SPMMs. To synchronize the granularity of data fetching with the memory access pattern, a novel memory-aware compressed format is proposed for sparse encoding to further reduce the memory transaction and the decoding overhead of C^3ache. To realize C^3ache, an efficient multi-precision matrix multiplication PIM macro is also implemented. Experimental results demonstrate that C^3ache achieves 7.22× and 2.4× speedup on average with 42.4× and 22.9× EDP reduction over the GPGPU CUDA Cores and Sparse Tensor Cores respectively, while reducing memory access more than 80%.

CCS Concepts

• **Computer systems organization → Single instruction, multiple data**.

Keywords

Processing-In-Memory, SPMM, GPGPU, Hybrid Dataflow, Cache Hierarchy

ACM Reference Format:
Xiaojie Li, Mingyu Wang, Baiqing Zhong, Haiqiu Huang, Guangjie Cao, and Zhiyi Yu. 2025. C3ache: Towards Hierarchical Cache-Centric Computing for Sparse Matrix Multiplication on GPGPUs. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3725843.3756077

1 Introduction

Sparse matrix multiplications (SPMMs) are fundamental kernels of complex operations in domains such as graph analytics [18, 30, 42, 51], deep learning [5, 10, 38], recursive formulations of all-pairs shortest-paths algorithms [13], peer pressure clustering [45], cycle detection [56], and matching algorithms [41]. It has also been widely used in scientific computing and engineering applications such as grammar parsing [39], chemical molecule dynamics [24], color intersection search [9], multigrid interpolation/restriction [8], Schur complement methods in hybrid linear solvers [53] and many other applications [6, 27, 54].

Given the importance of SPMMs, there are many proposals for domain-specific accelerators to accelerate them [4, 23, 26, 29, 36, 37, 48, 49, 55, 60]. Although these designs are notable successes, they are constrained by adaptability deficiency and memory isolation with CPUs. Thus, General-purpose graphics processing units (GPGPUs) have become pivotal computational platforms for executing SPMM due to their superior computing performance, programmability, and low task offload overhead. However, matrix sparsity exhibits significant diversity across applications. For instance, neural networks predominantly employ matrices with moderate sparsity

*Corresponding author: Mingyu Wang.

MICRO '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756077

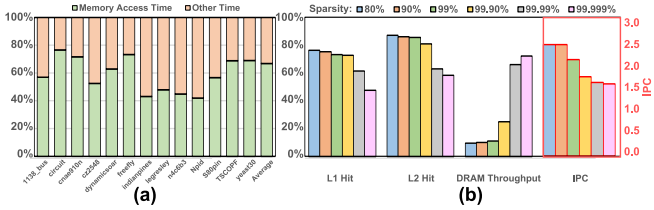


Figure 1: (a) Memory access overhead for various sparse matrices on RISC-V GPGPU. (b) Cache hit rate, memory throughput (%) and IPC degradation on NVIDIA A100 with cuSPARSE [35] (measured using Nsight Compute).

levels (e.g., 10%-90% non-zero density)[22, 47], whereas scientific and graph algorithms frequently handle ultra-sparse matrices with non-zero densities below 1% [14]. Thus, it is a challenge to efficiently execute SPMMs across varying sparsity regimes on GPGPUs.

To further enhance the performance of GPGPUs in matrix computations, prior studies have integrated specialized acceleration units into GPGPU processors, such as Tensor Cores (TCs [44]). These units dedicate nearly all their area to computation, achieving peak throughput on dense matrices (characterized by regular computation patterns and high arithmetic intensity). However, as matrix sparsity increases, their efficiency degrades rapidly due to redundant computations on zeros, which waste both time and energy. To mitigate this limitation, recent efforts (e.g., Sparse Tensor Cores, STCs [11, 40, 52, 61]) have extended TCs to handle sparse data by introducing distribution networks that gather unstructured or structured sparse inputs and reconfigurable accumulation buffers for sparse outputs. While these modifications improve performance at moderate sparsity levels, they remain suboptimal under high sparsity regimes.

These processor-centric optimizations often fall short in achieving efficient SPMMs across varying sparsity regimes on GPGPUs. From the perspective of optimizable space, the processor-centric optimizations of SPMMs on GPGPUs face the following critical bottlenecks:

❶ **Memory access overhead dominates computational overhead:** The irregular distribution of non-zero elements in sparse matrices disrupts spatial and temporal locality, causing inefficient cache utilization—each cache line loads only a limited number of valid data elements and triggers frequent cache line replacements. This irregular memory access pattern generates excessive memory transactions and latency [12, 19, 33], making it challenging for GPGPUs to effectively hide these delays. Although prior works mitigated memory traffic via tiling strategies, their optimization efficacy diminishes with increasing sparsity and scaling matrix dimensions. As illustrated in Figure 1, as sparsity increases, the pressure on memory access intensifies, which in turn leads to a degradation in IPC. This analysis reveals that the bottleneck for processing SPMMs resides in memory access constraints rather than computational throughput. Therefore, simply augmenting GPGPUs with specialized compute units fails to deliver significant performance gains. These units prioritize computational density over memory subsystem optimizations, making them ill-suited for sparse workloads dominated by irregular memory access and low arithmetic intensity.

❷ **The fundamental trade-off between data reuse efficiency and on-chip storage demands is difficult:** Current state-of-the-art matrix multiplication processor-centric implementations leverage three primary dataflow paradigms: inner product-based, outer product-based, and row-wise Gustavson's dataflow (detailed in Section 2.3). Among these, the inner product-based dataflow dominates matrix multiplication (MM) implementations due to its low on-chip memory footprint. However, this approach suffers from suboptimal data reuse, leading to excessive off-chip data movement. Conversely, the outer product-based dataflow achieves superior data reuse efficiency but imposes prohibitively high on-chip storage demands, which strain cache utilization and scalability. Gustavson's dataflow strikes a middle ground, offering moderate reuse and memory requirements. Low data reuse and high on-chip storage requirements will further exacerbate bottleneck ❶. The trade-off highlights an unresolved challenge: it is difficult to implement a dataflow that simultaneously maximizes data reuse while minimizing on-chip memory pressure in processor-centric architectures.

To address the aforementioned bottlenecks, this paper proposes C^3ache, an innovative hierarchical Cache-Centric computing solution for SPMMs on GPGPUs for the first time, which explores how to leverage the behavioral characteristics of SPMMs (memory access patterns and computational properties) and the memory hierarchy of GPGPUs to achieve near-optimal data reuse and memory efficiency. We first analyze the characteristics of different dataflows in SPMM regarding data reuse patterns and on-chip memory requirements. Building upon this analysis, we decouple the SPMM computation into two distinct phases with different behavioral characteristics (multiplication and merging phases) and align with memory access pattern based on the outer-product dataflow and Gustavson's dataflow. Furthermore, we systematically investigate the modern GPGPU memory architecture and data-sharing hierarchy across GPGPU cache levels, subsequently reconstructing the cache architecture through in-cache computing into dedicated processing units: large-scale data-parallel Processing-in-Memory (PIM) units and in-situ merging PIM units. This architectural transformation enables the mapping of SPMM's multiplication and merging phases to execute within the respective reconstructed cache hierarchies. Complementing this solution in software, we propose a novel memory-aware compressed format specifically optimized for sparse matrix encoding in SPMMs, which effectively reduces and balances memory transactions to further enhance the efficiency of our proposed solution. This provides a novel perspective on enhancing the execution efficiency of GPGPUs for SPMMs.

In summary, this paper makes the following contributions:

(1) We introduce a hybrid dataflow that synergistically integrates outer-product and Gustavson dataflow to decouple SPMM computation into two distinct phases: multiplication and merging. This dataflow operates at cache-line granularity tiling to align with memory access patterns.
(2) We propose the first hierarchical in-cache computing architecture that restructures GPGPU cache hierarchies into dual-mode PIM units: large-scale data-parallel PIM units and in-situ merging PIM units. This architectural transformation

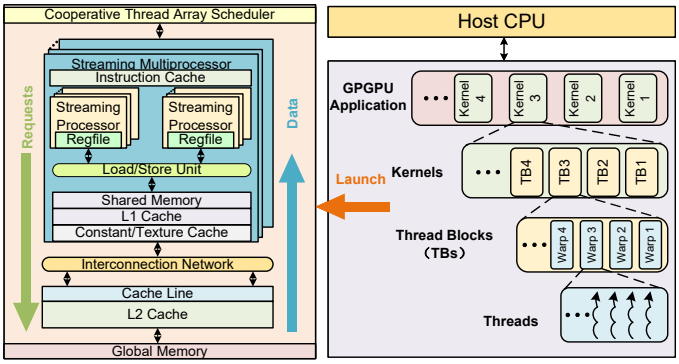


Figure 2: Modern GPGPU architecture and GPGPU application hierarchy

enables the mapping of SPMM's multiplication and merging phases to reduce the memory transactions and mitigate the overhead during shared variable updates in partial matrix merging. To ensure operational efficiency, a Stream-aware Request Management unit dynamically schedules input data prefetching and PIM requests through runtime data readiness monitoring.

(3) To support this solution in software, C3SR, a novel memory-aware compressed format, is proposed to specifically optimize for sparse matrix encoding in SPMM. C3SR enables the detection of cache line slack and facilitates the compression of multiple short sparse matrix rows into a single cache line to reduce the memory transaction.

(4) The efficient muti-precision matrix multiplication SRAM PIM macro (MSPM) is introduced to implement C^3ache. MSPM employs an exponent-mantissa pipelining technique to enhance processing performance and throughput, along with the scheme of in-memory booth encoding to minimize mantissa computation cycles and bitwise multiplication.

2 Backgroud and Motivation

2.1 Modern GPGPU Processor-Centric Architecture

As illustrated in Figure 2, contemporary GPGPUs integrate massively parallel compute architectures through clustered streaming multiprocessors (SMs), each comprising multiple streaming processors (SPs, termed CUDA Cores in NVIDIA architectures). These SMs employ hierarchical memory organization: threads within an SM share low-latency L1 cache and software-managed shared memory resources, while all SMs collectively access unified L2 cache through on-die interconnect. Most of mainstream GPGPUs adopt the cache topology in which LLC is larger than MLC. For example, a total of 108 SMs are used in NVIDIA A100 [11] with 192KB L1 on each SM and the SMs share 40MB L2. Generally, lower-level memory accesses incur higher latency and performance overhead. In the cache hierarchy of a GPGPU, data is accessed at the granularity of cache lines, typically 128 bytes.

GPGPUs exploit data-level parallelism through the Single Instruction Multiple Thread (SIMT) execution model [1], where each of these threads can follow its unique execution path and access any memory location. GPGPU applications are orchestrated through a hierarchical execution model. The host CPU initiates computational workloads comprising multiple kernels that are distributed across the entire GPGPU device for parallel processing. Within this hierarchy, the global scope of the GPGPU encompasses all concurrent threads executing on the device, which collectively share coherent access to the L2 cache. The kernel further organizes threads into thread blocks (TBs), with each TB constituting a fundamental scheduling unit. At the TB scope, threads are guaranteed colocation within the same SM, enabling shared access to the SM's L1 cache resources. The theads of TBs are divided into warps for parallel execution.

Modern GPGPU architectures exhibit processor-centric characteristics, where data processing happens in the processor and all memory requests are initiated by the processor [34]. Under extreme conditions, memory requests traverse the entire memory hierarchy to retrieve data from global memory. Conversely, fetched data ascend through the same hierarchy before becoming accessible for computation. This bidirectional traversal introduces substantial access latency inherent to hierarchical memory systems. While warp scheduling mechanisms partially mitigate this limitation by overlapping memory operations and computation across adjacent warps, their efficacy diminishes drastically for sparse matrix workloads. Specifically, the irregular memory access patterns and ultralow arithmetic intensity characteristic of sparse matrices exacerbate pipeline stalls, as the limited computational workload fails to amortize the prolonged latency periods. This architectural mismatch highlights that processor-centric designs struggle to decouple computation from rigid memory access dependencies.

2.2 In-Cache Computing for SPMMs on GPGPUs

According to the above analysis, high memory access overhead is a key factor that constrains the performance of SPMM on processor-centric architecture. PIM architectures [25, 28, 31, 57] present a transformative solution for mitigating memory access bottlenecks in GPGPUs through the convergence of memory and compute resources. By eliminating the von Neumann demarcation between computation and storage, PIM achieves radical reductions in data movement costs, thereby enhancing processor energy efficiency and performance.

Cache hierarchy are positioned as intermediaries between processors and global memory, which can serve as execution backends for processors and request frontends for global memory. Therefore, in-cache computing has emerged as an efficient PIM implementation. For instance, prior works [2, 3, 16, 17, 46, 58, 59] have explored in-cache computing for specific workloads. Neural Cache [16] proposes bit-serial computing to implement fixed-point addition, multiplication and reduction operations for neural networks inference. MAICC [17] integrates computing cache into many-core architecture for multi-DNN model inference. EVE [3] reconfigures the conputing cache into vector registers to implement RISC-V vector extensions.

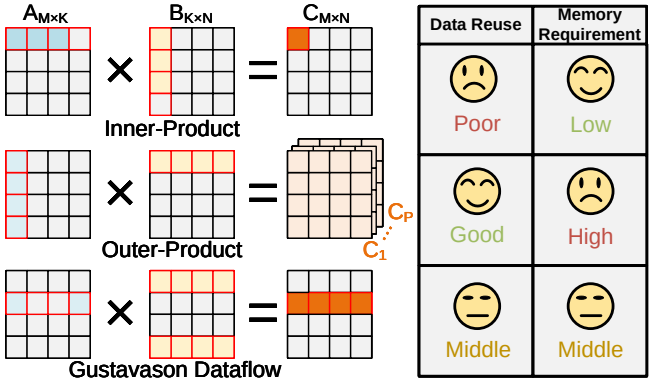


Figure 3: Overview of SPMM dataflows with different characteristics

However, existing in-cache computing designs predominantly focus on single-level cache. Processor-side implementations (L1-cache) [17, 58] enable fine-grained control over parallel computation streams, yet suffer from expensive overheads in shared variable synchronization due to the limited scope of shared variable visibility (restricted to TB-level sharing). Conversely, global-memory-side designs (Last-level cache, LLC) [3, 16, 59] alleviate shared variable update costs through broader-scoped synchronization domains but sacrifice computational agility. The characteristics of SPMM exacerbate this trade-off: its irregular parallelism demands instruction-level control over sparse parallel computation streams, while its memory-bound characteristic necessitates low-latency shared state updates (partial result accumulation). This fundamental mismatch motivates our hierarchical in-cache computing architecture (detailed in Section 3).

2.3 SPMM Dataflows

Figure 3 shows the three basic SPMM dataflows distinguished by the granularity of sparse reductions, which is a fundamental operation of SPMMs. Sparse reductions operate through three phases: sparse gather from a given index, apply an associative reduction operation to the index's value, and sparse scatter to the index. The three SPMM dataflows execute sparse reductions at the granularity of elements, rows, and partial product matrices respectively. This results in distinct characteristics for each dataflow, which will be analyzed in detail in this section.

The MM performs the computation $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, where $\mathbf{A} \in \mathbf{R}^{M \times K}$, $\mathbf{B} \in \mathbf{R}^{K \times N}$, and $\mathbf{C} \in \mathbf{R}^{M \times N}$. We define data reuse efficiency as the ratio of multiply-accumulate (MAC) operations to memory accesses($\eta = \frac{MAC\_Operations}{Memory\_Accesses}$). Let nnz denote the number of non-zero elements in a matrix. Under parallel computing paradigms, let $\mathbf{P}$ represent the hardware concurrency degree (number of parallel processing units).

2.3.1 ***Inner Product Dataflow.*** Conventional implementations of MMs predominantly employ the inner product dataflow, where each element $c_{i,j}$ is computed as:

$$c_{i,j} = \sum_{k=0}^{K-1} a_{i,k} \times b_{k,j}$$

This dataflow generates the element of the final matrix through row-column vector dot products across the operand matrices, performing index matching and executing at most $\frac{nnz}{M}$ MAC operations. The dataflow yields $O(\frac{nnz/M}{nnz/M+nnz/N})$ data reuse efficiency while imposing $O(P \times (nnz/M + nnz/N + 1))$ on-chip memory footprint. Due to $\frac{nnz/M}{nnz/M+nnz/N} < 1$, the inner product dataflow demonstrates suboptimal data reuse efficiency, indicating memory-access-bound characteristics of this dataflow paradigm. Nevertheless, cache requirements are favorable.

2.3.2 ***Outer Product Dataflow.*** The outer product dataflow decomposes matrix multiplication into rank=1 partial product matrices C_k through outer product operations between column vectors $a_{:,k}$ of matrix A and row vectors $b_{k,:}$ of matrix B, expressed as:

$$\mathbf{C} = \sum_{k=0}^{K-1} C_k = \sum_{k=0}^{K-1} a_{:,k} \times b_{k,:}$$

Outer product dataflow produces meaningful outputs for each pair of non-zero elements from a column of A and a row of B, excuting $(nnz/K)^2$ MAC operations and eliminating the need for index matching. Additionally, all elements in a row of B are shared for all elements in the corresponding column of A in outer product dataflow, which maximizes data reuse ($\eta = O(\frac{(nnz/K)^2}{2 \times nnz/K}) = O(\frac{nnz}{2K})$, typical ranges of $\frac{nnz}{2K}$ from 10-1000). Once the computation between a column of A and a row of B is complete, these elements are no longer used and can be evicted from the cache.

While the outer product dataflow enhances data reuse for SPMMs on GPGPUs, its performance is constrained by critical trade-offs. The computation of outer product dataflow generates many partial product matrices, which cause many times memory writes for final output (vs. 1 direct write in inner product) and redundant storage ($O(P \times (2 \times nnz/K + M \times N))$ on-chip memory footprint).

These partial product matrices generated by outer product dataflow will be stored in memory before merging, leading to extensive memory accesses and risks of evicting useful data from the cache, thereby degrading cache performance. This analysis reveals a challenge: while outer product optimization improves input data reuse, it simultaneously degrades output locality and cache performance, fundamentally limiting its efficacy for SPMMs on GPGPU architectures. Fortunately, the emergence of PIM technology has made it possible to balance data reuse and cache efficiency. This will be detailed in Section 3.

2.3.3 ***Gustavson Dataflow.*** The Gustavson dataflow (commonly termed row-wise product dataflow) operates through row-wise operand alignment: all non-zero elements from a row $a_{m,k}$ in matrix A are multiplied with corresponding non-zero entries from the rows $b_{k,:}$ of matrix B, where the target row index in B is determined by the column index of the originating non-zero in A. These partial products are subsequently accumulated into the corresponding output row $C_{m,:}$ of matrix C, expressed as:

$$\mathbf{C}_{\mathbf{m},:} = \sum_{k=0}^{K-1} a_{m,k} \times b_{k,:}$$

Analytically, each execution step performs $\frac{nnz}{M} \times \frac{nnz}{K}$ MAC operations and has $\eta = \frac{nnz/M \times nnz/K}{nnz/M + nnz/M \times nnz/K} = \frac{nnz}{K+nnz}$ data reuse, while requiring $O(P \times (\frac{nnz}{M} + \frac{nnz}{M} \times \frac{nnz}{K} + N))$ on-chip memory requirements.

Our comparative analysis reveals that Gustavson dataflow has moderate data reuse ($\frac{nnz}{K+nnz} \approx 1$) and memory footprint. Meanwhile, Gustavson dataflow exhibits architectural compatibility with GPGPU memory subsystems - the row based input/output patterns naturally align with cacheline-granular memory transactions (64/128-byte bursts in modern architectures). These characteristics form the foundation for our hybrid dataflow design (detailed in Section 3). However, Gustavson Dataflow introduces a significant degree of random access for Matrix B rows, which becomes increasingly pronounced as the matrix dimensions grow. Specifically, as the matrix dimension K increases, data reuse efficiency degrades proportionally. Additionally, variations in the number of nonzero elements per row in Matrix A lead to workload imbalance.

3 Methodology

3.1 Key Insights of C^3ache

To tackle the problem of heavy memory access overhead and poor data reuse when executing SPMM on GPGPUs, hierarchical in-cache computing and hybird product dataflow are both employed in C^3ache to achieve optimal data reuse and maximize memory efficency. The implementation of C^3ache is based on the following insights:

The SPMM computation has two distinct phases: the multiplication and merge phases. The multiplication phase is executed before the merge phase and involves data sharing and reuse across parallel computation streams. And the multiplication streams have consistent control flow. In contrast, the merge phase requires updating and synchronizing the partial product matrices generated by multiplication streams on a shared memory. The discrepancy leads to sub-optimal execution of the entire SPMM on single-level in-cache computing architecture.

In the GPGPU memory hierarchy, memory access requests flow from L1 cache to last level cache (LLC), while the data flows in the opposite direction. To reduce the cross-layer movement of requests and data, most memory access requests should be resolved within L1 cache, while avoiding the upward migration of infrequently used data from LLC.

Therefore, two computation phases of the SPMM are decoupled and mapped to different levels of cache separately on C^3ache. Executing the multiplication phase in L1 cache can satisfy the data reuse requirements of each computation stream and prevent the influx of large-scale memory access requests into LLC. Meanwhile, performing the merge phase in LLC can leverage its shared characteristics to efficiently update and synchronize the partial product matrices while avoiding the upward migration of partial product data.

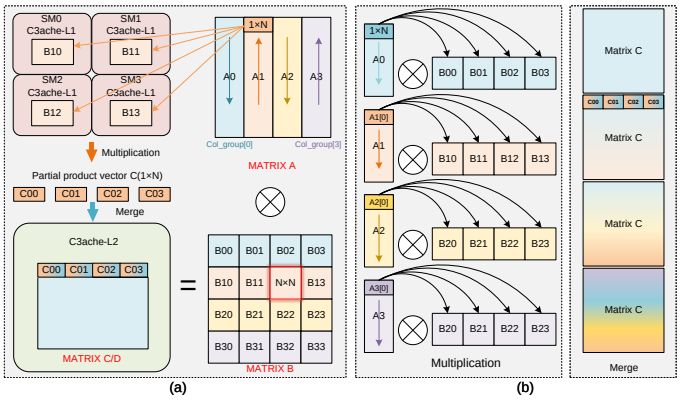


Figure 4: Hybrid product dataflow on C^3ache. (a) Mapping of hybrid product dataflow. (b) Two phases of hybrid product dataflow

3.2 Hybrid Dataflow of C^3ache

The qualitative analysis presented in the Section 2.3 elucidates the trade-offs between data reuse and memory requirements across three SPMM dataflow. To resolve this mismatch, we introduce a hybrid dataflow with cache-line granularity that synergistically combines the strengths of outer-product and Gustavson dataflow. This dataflow decouples SPMM computation into two distinct phases: multiplication and merging, while implementing cacheline-optimized tiling to align with modern memory transaction patterns.

The Gustavson dataflow is incorporated to align with GPGPU memory transaction patterns, where its row-based operand organization demonstrates compatibility with cacheline-granular memory access patterns. Specifically, the input phase generates sequential memory access pattern through row-wise traversal of sparse matrix A, matching cacheline boundaries in GPGPU architectures. Concurrently, partial product accumulation directly maps to full-cacheline writes in output phase.

Meanwhile, the outer-product dataflow is employed to eliminate redundant operand fetches and obtain the perfect data reuse. The outer-product dataflow inherently decouples SPMM into two computational phases, which is a decomposition that aligns with divide-and-conquer implementation strategies. This intrinsic phase separation enables independent optimization of multiplication parallelism and merge-phase synchronization.

As exemplified in Figure 4a ($D = A \times B + C$ as an example), the proposed dataflow partitions matrix A into column groups (Col_groups) of size N, where $N = \frac{cacheline\_size}{data\_size}$, while matrix B is tiled into $N \times N$ submatrices. For sparse rows that cannot fully occupy a cache line due to insufficient data density, the C3SR coalesces multiple sparse rows into a single cache line, with detailed implementation specifics provided in Section 3.4. Each Col_group of A calculates with identically indexed (color-coded) submatrices in matrix B. Each SM is mapped to a submatrix of matrix B, which can be readily accomplished through thread block allocation. During execution, rows of $A's$ Col_groups are broadcast sequentially at cacheline granularity to all SMs in a zigzag pattern. Each SM computes the product between a Col_group of matrix A and a B

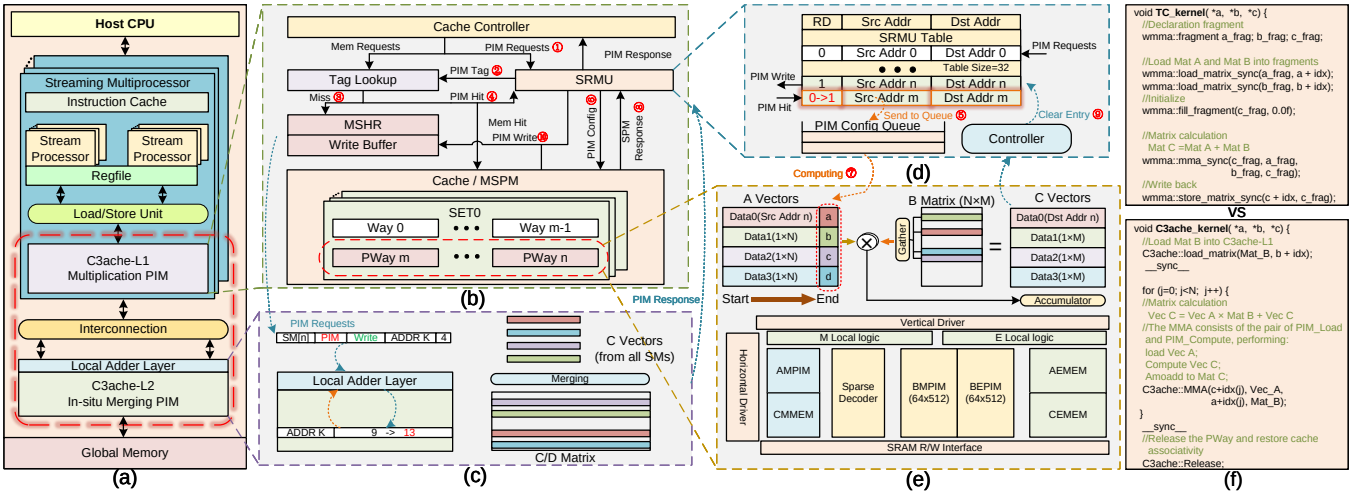


Figure 5: C^3ache architecture and PIM request excution flow. (a) Overview of C^3ache in GPGPU. (b) C^3ache-L1 structure. (c) Example of C^3ache-L2 processing PIM requests behavior. (d) Details of SRMU. (e) Details of MSPM. (f) Programming Example

submatrix, with a collaborative partial product matrix across SMs generating intermediate results for matrix C, as shown in Figure 4b. Following the computation of an $A's$ Col_group, SMs advance to subsequent B submatrices.

The C^3ache-L1 leverages Gustavson's dataflow to compute partial product vectors, which are then routed to C^3ache-L2 for in-situ merging via address-indexed accumulation. This hierarchical approach eliminates most of partial matrix transfers by confining merge operations within L2, thereby circumventing redundant memory traffic except for initial matrix fetches. Furthermore, each SM is allocated an equal number of computational tasks that traverse and process all col_groups of matrix A. The cacheline-grained broadcast of $A's$ Col_groups inherently balances computational workloads and memory traffic across SMs. This design exhibits native compatibility with existing GPGPU memory optimization schemes, including request coalescing, while maintaining cacheline-aligned access patterns.

Within this dataflow, the on-chip storage requirements are decoupled with the number of parallel compute streams P (maintaining a single instance of matrix C with the nnz_C storage requirements). This dataflow achieves maximal data reuse while maintaining bounded storage overhead by adjusting the size of the matrix C block to align with L2 cache capacity. Furthermore, the column-group partitioning of matrix A at cache-line granularity confines row fetch operations for matrix B within a bounded address range. From a memory perspective, the dataflow pattern is both streamlined and efficient.

3.3 C^3ache Microarchitecture

The C^3ache architecture depicted in Figure 5a is integrated within the RISC-V-based GPGPU microarchitecture. The baseline architecture comprises clustered SMs, each equipped with a private L1 cache and sharing an L2 cache with other SMs at the same hierarchical level. The unified L2 cache interfaces with a large off-chip global memory (DRAM). To address the dual-phase computational requirements of our hybrid dataflow, C^3ache architecturally restructures the cache hierarchy. The L1 cache is transformed into large-scale data-parallel PIM units optimized for multiplication phase, supporting efficient muti-precision matrix multiplication through bitline computing. The L2 cache is redesigned as in-situ merging PIM modules leveraging partial product matrix locality for atomic accumulation, achieving high merge operation concurrency through bank-level parallelism. This architectural design mitigates the expensive overhead of memory access by localizing the majority of matrix operations within the cache hierarchy while reducing partial product storage redundancy.

In addition to traditional cache structures, C^3ache-L1 integrates two specialized components: a Stream-aware Request Management Unit (SRMU) and MSPM, as depicted in Figure 5b. Due to employing the original instruction front-end pipeline, this design preserves baseline cache functionality without impeding the performance of other operations while enabling dual-mode operation (cache and PIM) through cache partitioning. C^3ache-L1 dual-mode ways are designed with MSPM, where standard SRAM function is maintained for read/write operations. The mode conversion is achieved via a dedicated PIM flag bit that dynamically partitions C^3ache-L1 into Data Ways and PIM Ways (PWays). Specifically, the C^3ache-L1 incorporates a dedicated 1-bit discrimination flag per dual-mode way to differentiate between Pways and Data Ways. When activated by PIM instructions, C^3ache-L1 sets the PIM flag and reduces cache associativity through modified LRU counter logic that excludes P-ways from replacement decisions. After completing the computation, the cache associativity can be restored by PIM flag reset. To maintain limited performance degradation of normal memomry access during cache partitioning, we impose a hardware-enforced constraint limiting Pways to $\leq$ 50% of total L1 capacity. The actual Pways ratio is configured at runtime through matrix tiling dimensions and parallelism, both of which are defined in software.

MSPM can simultaneously support the multiplication of four vectors from a sparse matrix A of size N with a matrix B of size

$N \times N$, as shown in Figure 5e. The corresponding rows of matrix B are selected for computation based on the column indices of the vector data. The operation is facilitated by the inherent 2D mesh structure of the PIM array that process the vertical accumulation of partial products via wordline activation of matrix B rows, rather than relying on the complex gather network.

The SRMU is designed to perform three critical functions for efficiently managing the PIM request life cycle at runtime, as shown in Figure 5d. Primarily, it stores the source and destination addresses of incoming PIM requests and maps source addresses to PWay indices. The SRMU comprises 32 entries for PIM requests, aligned with the GPGPU's thread warp configurations. This enables the SIMT pipeline to seamlessly offload tasks to C^3ache without causing pipeline stalls. Concurrently, the SRMU monitors the readiness of the PWay corresponding to the source address of the request. This facilitates the prioritization of computation-ready cache lines for the MSPM. Following computation, the SRMU initiates a store request with the result vector to the destination address while simultaneously clearing the corresponding completed PIM request entry from the SRMU table. Upon receiving the PIM store response, the SRMU forwards the response to the load-store unit (LSU) via the cache controller.

Table 1: SUMMARY OF ISA AND APIs

Instructions	Direction from Source to Destination
C^3ache.PIM_Load.type frag	Global Memory to C^3ache-L1
C^3ache.PIM_Compute.type des	C^3ache-L1 to C^3ache-L2 & Global Memory
C^3ache.PIM_Release	C^3ache-L1 to C^3ache-L1
Programming interface	**Description**
C^3ache::Load_matrix()	Load data (Matric B) from global memory to C^3ache-L1 and set the Pways
C^3ache::MMA()	Load data (Vector A), perform $D = A \times B + C$ and in-situ store data to C^3ache-L2 & global memory
C^3ache::Release()	Release the PWay and restore cache associativity

To maintain architectural compatibility while adapting the inherently serial nature of merge operations, C^3ache-L2 implements a minimally invasive hardware modification: a peripheral local adder layer integrated with existing cache banks. This design preserves original memory structures while enabling in-situ accumulating partial product vector from multiple SMs. As depicted in Figure 5c, C^3ache-L2 employs the write-back strategy and the PIM request is behavior as write request. the adder layer merges partial product vectors from distributed SMs during write-back operations. On cache hits, requested data undergoes accumulation with destination operands within the adder layer and is written back to destination location. For cache misses, destination operands is first fetched from global memory into L2 before proceeding with the merge-and-update sequence, ensuring operational integrity. In-situ accumulation within L2 cache ensures that the partial product matrices remain as much as possible in L2 cache, thereby avoiding the significant overhead associated with cross-level accesses. This approach also eliminates the need for multiple copies of partial product matrices, reducing the storage overhead for these matrices.

The behavior of PIM requests is illustrated in Fig. 5bcd. The tiled matrix B is preloaded into the MSPM. When executing the PIM instruction, the SIMT pipeline decodes the instruction and generates the PIM request, which is forwarded to C^3ache via the LSU. This request includes the source address (Src-Addr) of the vector from the sparse matrix A and the destination address (Dst-Addr) in matrix C, where the computed vector will be stored. Upon receiving the PIM request, C^3ache-L1 performs Request Registration in the SRMU and checks whether the data at Src-Addr is ready (❶-❸). For ready data, the request is allocated to the MSPM for computation, depending on the availability of resources in the MSPM (❹-❼). Once the computation is completed, the resulting vector is sent by the SRMU to initiate a PIM write request to the write buffer, and the request entry in the SRMU is cleared (❽-❿). Upon receiving the PIM write response, the cache controller forwards the response to the LSU.

C^3ache incorporates cache coherence considerations. Pway locking/release operations ensure that source data remain unmodified throughout execution. Computed results undergo atomic merge and modification exclusively within the globally shared L2 cache while leveraging L2's unified memory visibility for coherence across all SMs without explicit synchronization ordering.

To seamlessly integrate the C^3ache architecture, we have developed RISC-V ISA extensions and corresponding APIs encompassing cacheline reconfiguration, PIM computation, and resource deallocation, as shown in Table 1. This comprise three key operations: *PIM_Load* is employed to load data from the source address and assign it to PWays for locking. *PIM_Compute* complete the PIM process for the validated PWays, performing atomic accumulate operation of the results to the destination address directly within the L2 cache. Finally, *PIM_Release* is used to free the PWay and restore cache associativity after the entire PIM process is complete. The three instructions are vector instruction extensions operating at the cacheline granularity. In the SIMT pipeline of GPGPUs, their behavior is consistent with that of regular load/store operations. The critical differentiation occurs at the LSU, where request generation embeds PIM-specific metadata within the *Request Type* field (a universally implementation in modern GPGPU, such as load/store/atomic...). Consequently, the C^3ache is exclusively governed by memory requests generated through the LSU, deliberately avoiding structural modifications to the SIMT microarchitecture. This design philosophy ensures facilitation of integration with modern GPGPU. Figure 5f demonstrates the programming paradigm for invoking the C^3ache API, where the C^3ache::MMA executes vector-matrix multiplication and enable arbitrary-dimensional matrix operations through iterative tiling and thread collaboration, which adapts to irregular sparsity patterns in SpMM workloads. This contrasts with Tensor Core's(TC) rigid data dimensionality constraints. C^3ache-L1 can be efficiently scaled via replication of the MSPM hardware. C^3ache-L2 allows further scalability by increasing the throughput of local adders. Together, these features enable C^3ache to achieve high scalability with modest hardware overhead.

3.4 Memory-aware Compressed Format

Traditional sparse encoding formats, such as Coordinate (COO), Compressed Sparse Row (CSR), ELLPACK (ELL), and Diagonal (DIA), fall short of C^3ache's requirements for efficient SPMM across diverse sparsity patterns. COO represents nonzeros via independent (row, column, value) triplets, but suffers from the twin random-access bottleneck. CSR uses a row-pointer plus column-index scheme to enable row-level parallelism and is widely adopted for SPMM, yet

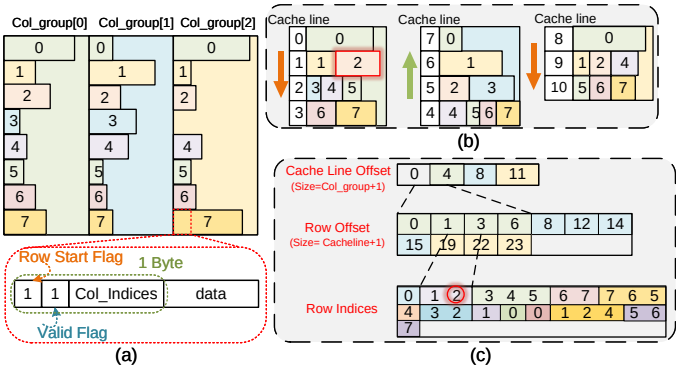


Figure 6: Example of C3SR compressed format (a) Row compression for each Col-group. (b) Compression of Multiple short rows into a single cache line. (c) Three groups of values in C3SR

it yields severe load imbalance on matrices with uneven row densities and offers limited optimization for column-dense workloads. ELL aligns each row to a fixed length for highly vectorized execution, but its compression and runtime efficiency collapse when nonzero counts vary significantly between rows. DIA stores data along specified diagonals for contiguous memory access, yet it is only suitable for diagonally dense matrices. Crucially, none of these representations exploit cache-line granularity or the memory-access characteristics necessary to coalesce nonzeros into single cache lines. A naive adoption of these formats may fail to fully capitalize on C^3ache's inherent performance advantages.

To support this solution in software, we propose C3SR, a novel memory-aware compressed format. C3SR facilitates the compression of multiple short sparse rows into a single cache line, using the cache-line as the indexing unit. This format synchronize the granularity of data fetching with the memory access pattern, which reduces the number of global memory transactions, addressing a limitation that most other sparse formats fail to resolve.

Relative positional information is inherently embedded between cache lines and PIM arrays. For a matrix tiled into blocks of cache line size, elements from the same row will always reside within the same cache line. This approach allows us to store only the relative column index of each element within a row, along with the one-row index, thereby capturing the full positional information of the elements efficiently.

As shown in Algorithm 1 and Figure 6, we first apply row compression individually to each Col-group, and then couple the relative column positions with the element of each row. The column index is encoded using additional 1 byte, with the first bit indicating whether the element is the first element of a new row, the second bit indicating whether the data is valid, and the remaining 6 bits representing a range from 0 to 64. This encoding scheme is sufficient to represent the relative column index for BF16, FP32, or INT8 data types when the tile size is equal to the cache line size.

To minimize unused space within each cache line, we compare the remaining space in the current cache line with the size of the next row to determine if they can fit into the same cache line. The

Algorithm 1 C3SR Compress Algorithm

Input: Sparse Matrix A in Dense Format
Output: Sparse Matrix A in C3SR

```
Row compression to each Col-group and record the number of non-zeros row_ nnz[colgroup][row]
for i = 0; i < colgroups; i++; j = 0; j <rows; j++ do
  if row _ nnz[i][j] != 0 then
    Row_Indices.pushbuck(j)
    if cacheline_slack >= row_nnz[i][j] then
      row_offset[CL] += 1;
      And cacheline_slack update;
    else
      cacheline_slack reset and update;
      row_offset.pushbuck(CL);
      CL += 1; %CL is cacheline indice
      Cache_Line_Offset[i+1] = CL;
    end if
  end if
end for
return Cache_Line_Offset; row_offset; Row_Indices;
```

short rows can be stored together within a single cache line if the capacity is enough. C3SR employs three distinct values to locate each row within a Col-group. The cache line offset records the position of each column group within the cache line, while the row offset indicates the position of the row within each cache line, and the row index stores the specific row indices for each row.

The process of generating the C3SR representation only requires traversing the elements of the original matrix, whether it is stored in COO or CSR format, thereby incurring limited overhead. Moreover, because the hybrid dataflow inherently ensures work balancing across each SM, C3SR does not require row reordering or other aggregation operations. Additionally, by encoding at the cache-line granularity, C3SR enables C^3ache to further ensure balanced memory access across each SM.

3.5 SRAM-PIM-based Implementation

To efficiently support sparse vector indexing and computation in C^3ache, we introduce the efficient multi-precision matrix multiplication SRAM-PIM macro (MSPM), which supports vector computations with BF16/FP32, and INT8 precision.

As shown in Figure 5e, MSPM includes four exponent (E) local logic units, four mantissa (M) local logic units, sparse decoder, the A mantissa PIM array (AMPIM) with 9T SRAM, the B PIM array (BEPIM,BMPIM) with 7T SRAM, other memory arrays (AMMEM, CMMEM, CEMEM) with 6T SRAM and the corresponding read/write and control circuits. BEPIM and BMPIM are connected to the E local logic and M local logic.

Due to the coupling of data and column indices, MSPM leverages the sparse decoder and the inherent 2D mesh structure of the PIM array to retrieve a row of matrix B (BEXP and BMAN) based on the column index and transfer it to the Local Logic. To efficiently compute floating-point data (e.g. BF16), MSPM decouples the exponent and mantissa and pipelines the computation. This enables the overlapping of computation for multiple rows with the gather

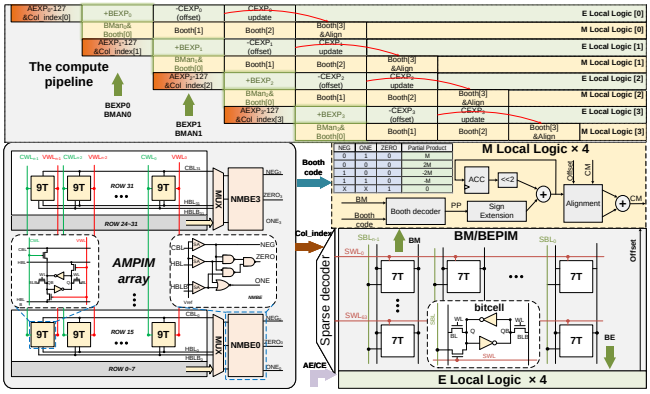


Figure 7: Detailed structure of MSPM and the compute pipeline

process, as well as the simultaneous execution of mantissa and exponent operations, as shown in Figure 7.

The exponent computation is performed in four stages: 1. The exponent of vector A (A_{EXP}) is first reduced by a bias of 127 and stored in a register. 2. The exponent of vector B (B_{EXP}) is then added to calculate the exponent part of $A \times B$ (denoted as AB_{EXP}). 3. AB_E is subtracted from the exponent of C (C_{EXP}), and the resulting offset is passed to the M local logic. 4. Based on the sign bit of the offset, a comparison is made between AB_{EXP} and C_{EXP}, and the value of C_{EXP} is updated in the CEMEM. In the mantissa computation, the scheme of in-memory Booth encoding is introduced to minimize the mantissa computation cycles and reduce bitwise multiplication complexity. In the AMPIM, a 9T SRAM is employed, with an additional transistor set controlled by the computation of the computing word line (CWL) and vertical WL (VWL). The Near Memory Booth Encoder (NMBE) encodes a 3-bit mantissa serially from the most significant bits (MSBs) to the least significant bits (LSBs). During the encoding process ($A_2A_1A_0$), both the CWL of cell A_2 and the VWL of cells A_1 and A_0 are activated. A_2 is read into the NMBE via the CBL, while an "AND" operation between A_1 and A_0 is performed on the horizontal bitline (HBL), along with a "NOR" operation on the inverted horizontal bitline (HBLB). The NEG, ZERO, and ONE signals obtained from Booth encoding are fed into the M local logic to indicate the corresponding operations for BM and perform shift-and-accumulate within the M local logic. After aligning the result of $A_M \times B_M$ with C_M, the values are added and updated in the CMMEM. Hence, MSPM efficiently support sparse vector indexing and computation in C^3ache.

4 Experimental Methodology

Table 2: GPGPU CONFIGURATIONS

SMs	32
Warp Size	32
Warps Scheduling	Round-Robin
L1 Data Cache	16 KB, 128 B line, 16 MSHRs, Write through, 8 ways ($Pways \leq 4$),
L2 Unified Cache	4 MB, 128 B line, 16 MSHRs, Write back, 16 ways

4.1 Experimental Setup

To analyze and evaluate the performance of C^3ache, we have adapted and modified a RISC-V GPGPU simulator, building upon Vortex 2.0 [50].

The simulator's architecture follows the one illustrated in Figure 5, integrating C^3ache for both the verifying its implementation and facilitating design space exploration. To accelerate simulation while preserving architectural proportionality, we implement scaled-down configurations based on the NVIDIA A100. The simulated system exhibits ~1/12 of the A100's peak performance (FP32 no tensor), with correspondingly scaled cache hierarchies. This proportional scaling methodology maintains consistent compute-to-memory ratios. The configuration parameters are detailed in Table 2. To maintain compatibility with the RISC-V compiler toolchain, we extended the RISC-V GPGPU with PIM operation instructions through the use of inline assembly. Building upon the hybrid dataflow outlined in Section 3.2, we implemented a parameterized SPMM kernel. To substantiate the efficiency of MSPM and obtain the parameters of MSPM for modeling C^3ache, we implemented the custom circuit of the MSPM and Local adder layer using Cadence Virtuoso. And the corresponding netlist was generated based on the TSMC 40nm process library, while performing simulations using Cadence Specter at the TT corner and 25°C.

4.2 Baselines

In order to fairly evaluate the performance benefits of C^3ache, we implemented the Vortex baseline (as CUDA core-based design) and sparse tensor core [61] (STC) within the same simulation environment on behalf of processor-centric optimizations. Specifically, according to [44], we have designed a hardware unit within the pipeline of Vortex that supports 64 multiply-accumulation (MAC) operations per cycle across 4 threadgroups as TC, and extend matrix load and store primitives. Using TC as the backbone, we replicated the STC through extension of the operand buffer and offset registers. The latency is aligned with [61]. The SPMM kernels are implemented in OpenCL based on clSPARSE [20].

Table 3: Low-sparsity Workloads

Models	Sparsity	Pruning method of Cuda Cores, STC, C^3ache	Compressed Format of Cuda Cores, STC, C^3ache
Resnet50-1	87.5%	Top-K, VectorSparse, Top-K	CSR, Vector-wise Encoding, C3SR
Resnet50-2	75%	Top-K, VectorSparse, Top-K	CSR, Vector-wise Encoding, C3SR
VGG16-1	87.5%	Top-K, VectorSparse, Top-K	CSR, Vector-wise Encoding, C3SR
VGG16-2	75%	Top-K, VectorSparse, Top-K	CSR, Vector-wise Encoding, C3SR
BERT	75%	Top-K, VectorSparse, Top-K	CSR, Vector-wise Encoding, C3SR
GPT2-1	53.76%	Null	CSR, Vector-wise Encoding, C3SR
GPT2-2	60.1%	Null	CSR, Vector-wise Encoding, C3SR
DeepSeek-1	88.7%	Null	CSR, Vector-wise Encoding, C3SR
DeepSeek-2	92.1%	Null	CSR, Vector-wise Encoding, C3SR

4.3 Datasets

To comprehensively evaluate C^3ache's support for different workloads, we collect workloads from both low-sparsity and high-sparsity matrices.

Low-sparsity Workloads: We use real-world deep neural network (DNN) workloads as representative low-sparsity matrix samples, with sparsity levels ranging from 50% to 92%, as shown in

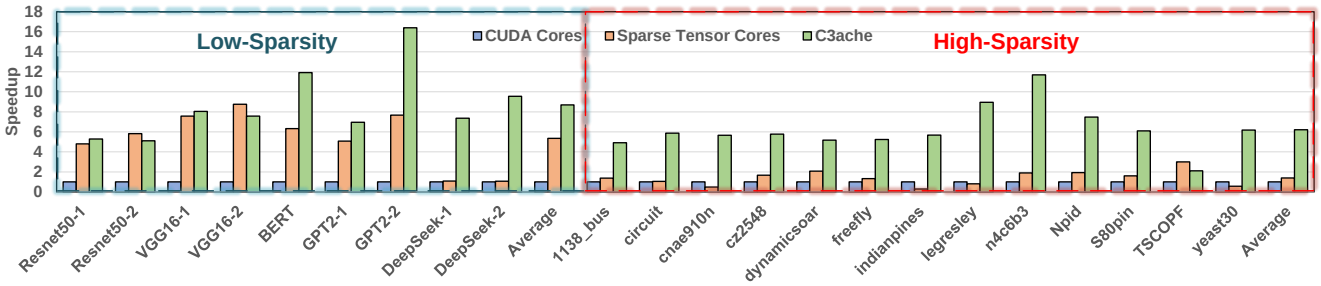


Figure 8: Speedup on CUDA cores, Sparse Tensor cores, and C^3ache.

Table 4: High-sparsity Workloads from SuiteSparse

Matrix	Dim	*NNZ*	Variance	Kind	Plot
1138_bus	1138 × 1138	2596	1.51	Power Network Problem	
circuit_2	4510 × 4510	21199	506.5	Circuit Simulation Problem	
cnae9_10NN	1080 × 1080	9139	76.9	Undirected Weighted Graph	
cz2548	2548 × 2548	25674	51.2	2D/3D Problem	
dynamic Soaring Problem_8	3543 × 3543	20064	35.1	Optimal Control Problem	
freeFlying Robot_2	1338 × 1338	6200	74	Optimal Control Problem	
indianpines _10NN	9144 × 9144	62328	18.5	Undirected Weighted Graph	
LeGresley _4908	4908 × 4908	30482	10.5	Power Network Problem	
n4c6-b3	5970 × 1330	23880	0	Combinatorial Problem	
N_pid	3625 × 3923	8054	38.1	Biochemical Network	
S80PI_n	4182 × 4182	10261	0.3	Eigenvalue/Model Reduction Problem	
TSC _OPF _1047	8140 × 8140	1012521	47434.9	Power Network Problem	
yeast_30NN	1484 × 1484	31175	174.2	Undirected Weighted Graph	

Table. 3. Specifically, we extract weight matrices from conventional DNN architectures such as ResNet-50 [22] and VGG-16 [47]. Additionally, we sample probability matrices from large-scale language models, including BERT [15], GPT-2 [43], and DeepSeek [21].

For both CUDA cores and C^3ache, we apply the Top-K pruning method to ResNet-50, VGG-16, and BERT to achieve varying sparsity levels, as this is a widely adopted pruning technique. ResNet-50 and VGG-16 undergo global Top-K pruning, while the BERT model is pruned using row-wise Top-K. For STC, we adopt the VectorSparse approach [61] to prune ResNet-50, VGG-16, and BERT to similar sparsity, which ignores the accuracy loss.

GPT-2 and DeepSeek, as generative models, inherently exhibit a lower triangular matrix structure with a natural sparsity. Therefore, we do not apply pruning but instead employ appropriate encoding techniques for compression.

High-sparsity Workloads: To assess the support of C^3ache for high-sparsity workloads with various characteristics, we adapt the real-world matrices derived from the University of Florida SuiteSparse Matrix Collection [14], as shown in Table. 4. The real-world matrices encompass a wide range of domains, including power network, circuit simulation, undirected weighted graph, 2D/3D Problem, optimal control problem, machine learning, combinatorial problem, etc. We choose matrices from this collection that have both regular and irregular structures. These matrices have dimensions varying from 1080 to 9144 and densities ranging between 0.074% and 1.528%. The variance in the number of non-zeros (NNZ) per row ranges from 0 to 47434.

For CUDA cores and C^3ache, we encode matrices using the CSR and C3SR formats, respectively. Since STC is inherently inefficient for high-sparsity SPMMs, even resulting in lower performance than CUDA cores, we first employ the BSR format [7] to eliminate tiles that consist entirely of zeros. In addition, we apply the VectorSparse to prune within tiles, ensuring that each row vector of the tile maintains a sparsity level exceeding 75%. Although such pruning is generally not acceptable in domains such as scientific computing due to its distortion of original data patterns, we adopt this approach to maximize the performance of STC and thereby demonstrate the superiority of C^3ache.

5 Experimental Results

5.1 Performance Evaluation

Table 5: COMPARISONS WITH OTHER WORKS

	ESSCIRC'22[25]	VLSI'21[31]	TCASI'23[28]	**This work (MSPM)**
Technology	28nm	28nm	65nm	**40nm**
Frequency(MHz)	53-403	250	200	**700**
Supply Voltage(V)	0.5-0.9	0.68-1.1	0.8-1.1	**1.1**
Throughput(GOPS,GFLOPS)	57.9-4.84(FP8-FP32)	119.4(BF16)	128(INT16)	**98.46(BF16) 196.92(INT8)**
Energy Efficiency(TOPS/W, TFLOPS/W)	1.644-0.099(FP8-FP32)	1.43(BF16)	1.23(INT16)	**2.28(BF16) 5.59(INT8)**

Energy efficiency: As shown in Table 5, our MSPM exhibits significant efficiency over prior research. Specifically, our design

achieves the energy efficiency of 2.28 $TFlops/W$ in BF16 calculations and 5.59 $Tops/W$ in INT8 calculations.

Speed up: To assess the cycle counts and speedup of SPMM with varying matrices, we executed low-sparsity workloads and high-sparsity workloads using CUDA Cores, STC, and C^3ache, as illustrated in the Figure 8. Experimental result demonstrates an average speedup of 8.69× and 1.62× over CUDA cores and STC for low-sparsity workloads. STC performs well when matrices exhibit regular structures. However, its performance deteriorates rapidly for irregular matrices, such as the lower triangular matrices found in Deepseek. For high-sparsity workloads, C^3ache achieves an average speedup of 6.21× and 4.46× over CUDA cores and STC. As sparsity increasing, STCs achieve only marginal performance parity with CUDA Cores. This limitation arises because the non-zero elements are insufficient to fully utilize the minimal parallel units, even when employing optimizations such as Block Sparse Row (BSR) encoding and VectorSparse pruning. In contrast, C^3ache consistently delivers robust support across diverse sparsity patterns. This improvement is attributed to the efficient data reuse in the hybrid dataflow for sparse input matrices, as well as the elimination of irregular memory accesses through hierarchical in-cache computing and cache-line level data compression with C3SR.

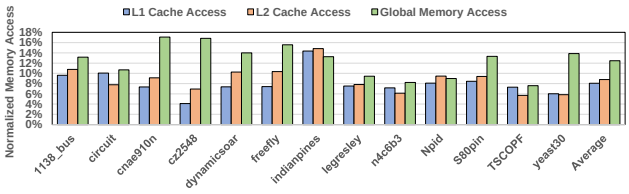


Figure 9: Memory access comparisons of C^3ache and CUDA cores (Normalized to CUDA cores).

Memory Access Reduction: To evaluate the optimization of C^3ache in reducing irregular memory accesses and redundant input fetches, we measured the reduction in memory requests at memory hierarchy, comparing C^3ache with the RISC-V GPGPU baseline while executing the same SPMM tasks, as illustrated in the Figure 9. The experimental results demonstrate that C^3ache reduced L1 accesses by an average of 91.9%, L2 accesses by 91.2%, and global memory accesses by 87.53%.

Power comparison: To ensure a fair comparison of the power consumption of the main data path between C^3ache, STC and the RISC-V GPGPU baseline, we study the runtime activities to calculate the dynamic power. The energy evaluation of C^3ache is based on power consumption data obtained from Cadence Spectre. The performance traces derived from RISC-V GPGPU simulator are sent to McPAT [32] to generate the power traces. As illustrated in Figure 10, C^3ache achieves 80.3% and 88.8% average energy saving over CUDA Cores and STCs respectively, alongside 42.3× and 22.9× reductions in Energy-Delay Product (EDP). These gains stem from its synergistic hardware-software optimization for maximizing memory efficiency and data reuse.

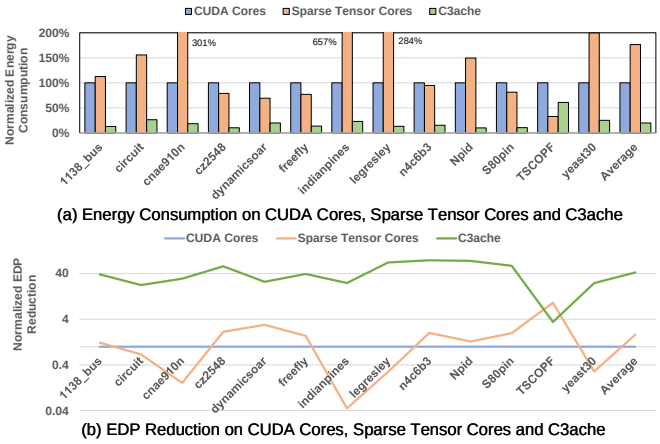


Figure 10: (a) Energy consumption and (b) EDP reduction comparisons of CUDA cores, Sparse Tensor cores and C^3ache (Normalized to CUDA cores).

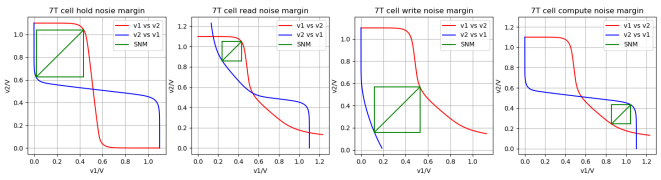
Table 6: DETAILED AREA BREAKDOWN

Component	AMPIM(9T)	B(EM)PIM(7T)	Write Driver	Sparse & Row Decoder	Total Area
Area (μm^2)	31071.505	128221.184	3676.16	2408.764	
Component	AEMEM(6T)	C(ME)MEM(6T)	Sense Amplifier	Precharge Circuit	312763.176 × 32 (C^3ache-L1)
Area (μm^2)	20606.976	10303.488	1296.243	1040.998	
Component	E Local Logic	M Local Logic	SRMU	Local Adder Layer	46528.2048 (C^3ache-L2)
Area (μm^2)	29142.221	74722.099	10273.536	46528.2048	

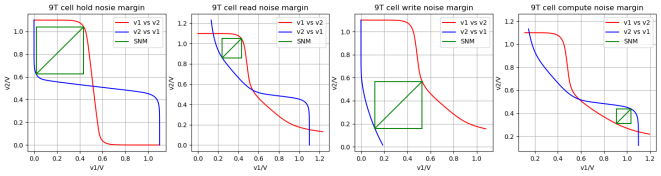
5.2 Design Overhead Analysis

Area: To evaluate the area overhead introduced by C^3ache on the baseline RISC-V GPGPU, we used the Cadence Virtuoso tool to construct MSPM and calculated the areas based on custom layouts and standard cell areas. The detailed hardware area breakdown of C^3ache is shown in Table 6. Meanwhile, we utilized the Synopsys Design Compiler tool and the TSMC 40nm standard cell library to synthesize the Vortex GPGPU. The evaluation results indicate that C^3ache introduces an area overhead of less than 2.5% to the baseline RISC-V GPGPU.

Stability: To evaluate the stability characteristics of C^3ache architecture, we designed and implemented a dedicated test circuit for static noise margin (SNM) quantification. Figure 11 illustrates the measured hold, read, write, and computing SNM characteristics of the MSPM. Controlled noise signals were injected at the complementary storage nodes (Q and QB) through programmable voltage sources (V1 and V2). The red trace denotes the variation of V1 relative to V2, the blue trace represents the variation of V2 relative to V1, and the green squares quantify the SNM boundaries of each operation. Experimental measurements demonstrate that the designed 9T SRAM and 7T SRAM have both a hold noise margin of 416 mV, a read noise margin of 189 mV, and write noise margin of 408mV. Due to the separation of read/write ports, and compute ports, the read, write, and hold noise margins of the MSPM are comparable to traditional 6T SRAM. During computation, the noise margin of the 7T SRAM for computation is 189 mV, which is similar to its read noise margin. The computation noise margin of the 9T SRAM is



(a) The Stablity Test Results of 7T SRAM



(b) The Stablity Test Results of 9T SRAM

Figure 11: The Stablity Test Results

128 mV, slightly lower than the read noise margin. However, this design performs computations in the digital domain using bitline computing, and the noise margin is sufficient to ensure the robustness of the results. These results validate that the proposed C^3ache configuration, achieved by augmenting the baseline 6T SRAM with additional transistors, maintains structural stability while enabling efficient matrix multiplication.

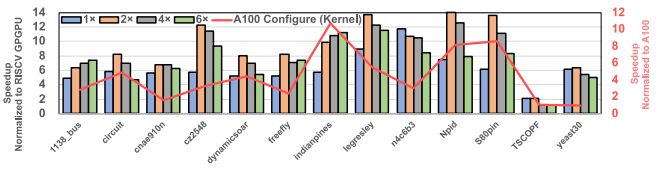


Figure 12: Speedup under different configurations

Scalability Analysis: To validate architectural scalability and performance sensitivity, we compare the performance of both C^3ache and the baseline under progressively scaled configurations (2×, 4×, 6× of cache configurations in Table 2) and illustrated in the Figure 12. Experimental results show that as the system scales, C^3ache consistently maintains strong performance. Even though the speedup slightly decreases on certain datasets, it quickly stabilizes. This is primarily because the baseline benefits from reduced memory pressure as the cache capacity increases. However, even when the L2 cache becomes large enough to fully hold datasets such as 1138_bus, C^3ache still delivers performance gains. This is attributed to C^3ache reduce memory traffic across all levels of the hierarchy. As a result, C^3ache achieves performance closer to the theoretical peak. Furthermore, we align the number of SMs and the memory size configuration with NVIDIA A100. We benchmark C^3ache against NVIDIA's commodity full-stack solution (STCs + 2:4 structured sparsity + cuSPARSELt, highlighted in red), excluding task offloading and data preprocessing overheads. C^3ache achieves about 4.4× speedup on average under normalized frequency conditions. These findings suggest that C^3ache exhibits robust performance scalability and maintains efficiency under large-scale configurations.

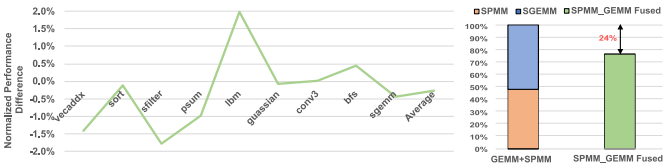


Figure 13: Performance difference compared with baseline RISC-V GPGPU (the left) and Collaboration capability with other kernels (the right)

Generality Analysis: To verify that the introduction of the C^3ache architecture does not compromise the general-purpose nature of GPGPU systems, we selected several commonly used GPGPU kernels for evaluation, as shown in the Figure 13. Experimental results demonstrate that, compared to baseline, integrating C^3ache incurs virtually no performance loss, with an average performance difference of only 0.2%. This is because when not explicitly invoked, C^3ache behaves like a conventional cache. Moreover, C^3ache converts only a portion of the cache ways into Pways rather than the entire cache, in order to preserve its ability to cooperate with other kernels. Relatively idle SIMT units can thus be utilized for compute-intensive kernels at the same time. An example of such cooperation between SGEMM and SPMM kernels, which is common in LLMs, is shown in the right of Figure 13. When SGEMM and SPMM are fused and executed concurrently on the C^3ache-enabled GPGPU (with disjoint data), the total execution time is reduced by 24%, and the performance is improved by 1.3 × compared to running the two kernels sequentially. These results confirm that C^3ache retains the general-purpose capability of GPGPUs while also enabling effective inter-kernel collaboration.

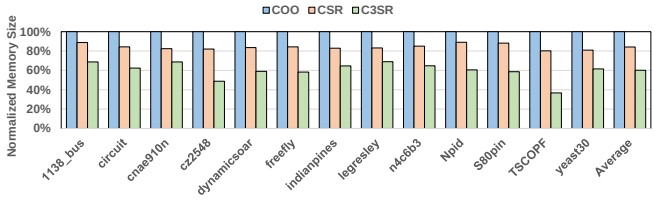


Figure 14: Memory overhead comparisons of COO, CSR and C3SR (Normalized to COO).

Memory overhead of C3SR format: To further assess the compression effectiveness of C3SR on sparse data, we measured the memory size occupied by the data encoded with C3SR and compared it with the COO and CSR formats, as illustrated in the Figure 14. The experimental results demonstrate that the memory size of C3SR is only 60.1% of the COO format and 71.41% of the CSR format on average. This improvement is attributed to the use of cacheline granularity in C3SR encoding, which allows the column index to be represented with just one byte.

Work-load Balance: To evaluate the workload balance efficacy of C^3ache, quantified by the imbalance ratio $\eta = (n_{max} - n_{min})/n_{max}$ where higher η indicates greater imbalance, we designed a controlled experiment under constrained execution conditions. Specifically, we enforced a single active warp per SM to isolate

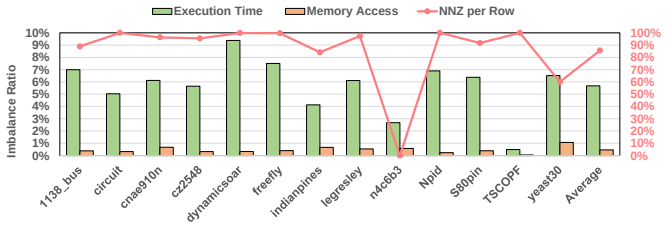


Figure 15: Imbalance ratio of execution time, memory access across and NNZ distribution.

the effects of C^3ache from potential optimizations introduced by warp scheduling dynamics. Under this configuration, we measured execution time imbalance and memory access imbalance across SMs. As demonstrated in Figure 15, the experimental results reveal that C^3ache exhibits excellent balance characteristics: execution time across SMs exhibited an average imbalance of 5.6%, while memory access patterns demonstrated near-uniform distribution with imbalance ratios of only 0.46%, even under conditions where the sparse matrix exhibits NNZ imbalance rate of 85.6%.

6 Conclusions

In order to mitigate the overhead of excuting SPMM across varying sparsity regimes on GPGPU due to irregular memory accesses and the redundant input fetches in the sparse matrix, we proposed C^3ache, a hierarchical cache-centric computing architecture with hybrid dataflow to acheive near-optimal memory efficiency and data reuse. The hybrid dataflow decouples the SPMM computation into two distinct phases and align with the memory access pattern. C^3ache restructures the cache hierarchy through in-cache computing, transforming the two levels of cache into distinct PIM units to map the two computation phases of SPMMs. and map the hybrid dataflow to different levels of the cache hierarchy. Additionally, C^3ache employs C3SR format to reduce the memory transaction and synchronize the granularity of data fetching with the memory access pattern. An efficient matrix multiplication SRAM-PIM architecture is also designed to implement C^3ache. Our study demonstrates that C^3ache achieves 7.22× and 2.4× speedup on average with 42.4× and 22.9× EDP reduction over the GPGPU CUDA Cores and Sparse Tensor Cores respectively, while reducing memory access more than 80%. Therefore, these advantages underscore the immense potential of C^3ache as a hierarchical cache-centric computing solution for the efficient execution of SPMMs on GPGPUs.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 92373103, 62204271, 62334014.

References

[1] Tor M Aamodt, Wilson Wai Lun Fung, Timothy G Rogers, and Margaret Martonosi. 2018. *General-Purpose Graphics Processor Architectures.* Springer.

[2] Shaizeen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. 2017. Compute Caches. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA).* IEEE, 481–492.

[3] Khalid Al-Hawaj, Tuan Ta, Nick Cebry, Shady Agwa, Olalekan Afuye, Eric Hall, Courtney Golden, Alyssa B Apsel, and Christopher Batten. 2023. EVE: Ephemeral Vector Engines. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA).* IEEE, 691–704.

[4] Veerendra Allada, Troy Benjegerdes, and Brett Bode. 2009. Performance Analysis of Memory Transfers and GEMM Subroutines on NVIDIA Tesla GPU Cluster. In *2009 IEEE International Conference on Cluster Computing and Workshops.* 1–9. https://doi.org/10.1109/CLUSTR.2009.5289124

[5] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing Fine-Grained Parallelism in Algebraic Multigrid Methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.

[6] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing Fine-Grained Parallelism in Algebraic Multigrid Methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.

[7] Nathan Bell and Michael Garland. 2008. *Efficient Sparse Matrix-Vector Multiplication on CUDA.* Technical Report. Nvidia Technical Report NVR-2008-004, Nvidia Corporation.

[8] William L Briggs, Van Emden Henson, and Steve F McCormick. 2000. *A Multigrid Tutorial.* SIAM.

[9] Timothy M Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing.* 590–598.

[10] Timothy M. Chan. 2010. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. *Proceedings of the Annual ACM Symposium on Theory of Computing* 39 (2010), 2075–2089.

[11] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia A100 Tensor Core GPU: Performance and Innovation. *IEEE Micro* 41, 2 (2021), 29–35.

[12] Jason Cong, Zhenman Fang, Michael Lo, Hanrui Wang, Jingxian Xu, and Shaochong Zhang. 2018. Understanding Performance Differences of FPGAs and GPUs. In *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM).* IEEE, 93–96.

[13] Paolo D'alberto and Alexandru Nicolau. 2007. R-Kleene: A High-Performance Divide-and-Conquer Algorithm for the All-Pair Shortest Path for Densely Connected Networks. *Algorithmica* 47 (2007), 203–213.

[14] Timothy A Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Transactions on Mathematical Software (TOMS)* 38, 1 (2011), 1–25.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers).* 4171–4186.

[16] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaaauw, and Reetuparna Das. 2018. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *2018 ACM/IEEE 45Th annual international symposium on computer architecture (ISCA).* IEEE, 383–396.

[17] Renhao Fan, Yikai Cui, Qilin Chen, Mingyu Wang, Youhui Zhang, Weimin Zheng, and Zhaolin Li. 2023. Maicc: A Lightweight Many-core Architecture with In-Cache Computing for Multi-DNN Parallel Inference. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture.* 411–423.

[18] John Russell Gilbert, Steven K. Reinhardt, and Viral B Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science and Engineering* (2008).

[19] Georgios Goumas, Kornilios Kourtis, Nikos Anastopoulos, Vasileios Karakasis, and Nectarios Koziris. 2008. Understanding the Performance of Sparse Matrix-Vector Multiplication. In *16th Euromicro Conference on Parallel, Distributed and Network-Based Processing (PDP 2008).* IEEE, 283–292.

[20] Joseph L Greathouse, Kent Knox, Jakub Poła, Kiran Varaganti, and Mayank Daga. 2016. clsparse: A Vendor-Optimized Open-Source Sparse Blas Library. In *Proceedings of the 4th International Workshop on OpenCL.* 1–4.

[21] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. *IEEE* (2016).

[23] Reza Hojabr, Ali Sedaghati, Amirali Sharifian, Ahmad Khonsari, and Arrvindh Shriraman. 2021. Spaghetti: Streaming Accelerators for Highly Sparse GEMM on FPGAs. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA).* IEEE, 84–96.

[24] Satoshi Itoh, Pablo Ordejón, and Richard M Martin. 1995. Order-N Tight-Binding Molecular Dynamics on Parallel Computers. *Computer physics communications* 88, 2-3 (1995), 173–185.

[25] Sangsu Jeong, Jeongwoo Park, and Dongsuk Jeon. 2022. A 28nm 1.644 TFlops/W Floating-Point Computation SRAM Macro with Variable Precision for Deep Neural Network Inference and Training. In *ESSCIRC 2022-IEEE 48th European Solid State Circuits Conference (ESSCIRC).* IEEE, 145–148.

[26] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. 2018. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. *arXiv preprint arXiv:1804.06826* (2018).

[27] George Karypis, Anshul Gupta, and Vipin Kumar. 1994. A Parallel Formulation of Interior Point Algorithms. In *Supercomputing'94: Proceedings of the 1994 ACM/IEEE Conference on Supercomputing*. IEEE, 204–213.

[28] Hyunjoon Kim, Junjie Mu, Chengshuo Yu, Tony Tae-Hyoung Kim, and Bongjin Kim. 2023. A 1-16b Reconfigurable 80Kb 7T SRAM-Based Digital Near-Memory Computing Macro for Processing Neural Networks. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 4 (2023), 1580–1590.

[29] Jinkwon Kim, Myeongjae Jang, Haejin Nam, and Soontae Kim. 2023. HARP: Hardware-Based Pseudo-Tiling for Sparse Matrix Multiplication Accelerator. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1148–1162.

[30] Thomas N Kipf and Max Welling. 2016. Semi-supervised Classification with Graph Convolutional Networks. *arXiv preprint arXiv:1609.02907* (2016).

[31] Juhyoung Lee, Jihoon Kim, Wooyoung Jo, Sangyeob Kim, Sangjin Kim, Jinsu Lee, and Hoi-Jun Yoo. 2021. A 13.7 TFLOPS/W Floating-Point DNN Processor using Heterogeneous Computing Architecture with Exponent-Computing-in-Memory. In *2021 Symposium on VLSI Circuits*. IEEE, 1–2.

[32] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. 2009. McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures. In *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*. 469–480.

[33] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse Matrix-Matrix Multiplication on Modern Architectures. In *2012 19th International Conference on High Performance Computing*. IEEE, 1–10.

[34] Onur Mutlu. 2023. Lightning Talk: Memory-Centric Computing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–2. https://doi.org/10.1109/DAC56929.2023.10247896

[35] Maxim Naumov, L Chien, Philippe Vandermersch, and Ujval Kapasi. 2010. Cusparse Library. In *GPU technology conference*, Vol. 12.

[36] G Noble, S Nalesh, and S Kala. 2023. MOSCON: Modified Outer Product based Sparse Matrix-Matrix Multiplication Accelerator with Configurable Tiles. In *2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID)*. IEEE, 264–269.

[37] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. Outerspace: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 724–736.

[38] Gerald Penn. 2006. Efficient Transitive Closure of Sparse Matrices over Closed Semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81.

[39] Gerald Penn. 2006. Efficient Transitive Closure of Sparse Matrices over Closed Semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81.

[40] Jeff Pool, Abhishek Sawarkar, and Jay Rodge. 2021. Accelerating Inference with Sparsity using the Nvidia Ampere Architecture and Nvidia Tensorrt. *NVIDIA Developer Technical Blog, https://developer. nvidia. com/blog/accelerating-inference-with-sparsityusing-ampere-and-tensorrt* (2021).

[41] Michael O Rabin and Vijay V Vazirani. 1984. *Maximum Matchings in General Graphs through Randomization.* Center for Research in Computing Techn., Aiken Computation Laboratory, Univ.

[42] Michael O Rabin and Vijay V Vazirani. 1989. Maximum Matchings in General Graphs through Randomization. *Journal of Algorithms* 10, 4 (1989), 557–567.

[43] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language Models Are Unsupervised Multitask Learners. *OpenAI blog* 1, 8 (2019), 9.

[44] Md Aamir Raihan, Negar Goli, and Tor M Aamodt. 2019. Modeling Deep Learning Accelerator Enabled GPUs. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 79–92.

[45] Viral B Shah. 2007. *An Interactive System for Combinatorial Scientific Computing with an Emphasis on Programmer Productivity.* University of California, Santa Barbara.

[46] William Andrew Simon, Yasir Mahmood Qureshi, Marco Rios, Alexandre Levisse, Marina Zapater, and David Atienza. 2020. BLADE: An In-Cache Computing Architecture for Edge Devices. *IEEE Trans. Comput.* 69, 9 (2020), 1349–1363.

[47] Karen Simonyan and Andrew Zisserman. 2014. Very Deep Convolutional Networks for Large-Scale Image Recognition. *Computer Science* (2014).

[48] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. (2020).

[49] Guangming Tan, Linchuan Li, Sean Triechle, Everett Phillips, Yungang Bao, and Ninghui Sun. 2011. Fast implementation of DGEMM on Fermi GPU. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11.

[50] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. 2021. Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics. In *MICRO '21*. Association for Computing Machinery.

[51] Stijn Van Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, University of Utrecht* (2000).

[52] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-side Sparse Tensor Core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1083–1095.

[53] Ichitaro Yamazaki and Xiaoye S Li. 2010. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *International Conference on High Performance Computing for Computational Science*. Springer, 421–434.

[54] Ichitaro Yamazaki and Xiaoye S Li. 2010. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *International Conference on High Performance Computing for Computational Science*. Springer, 421–434.

[55] Yifan Yang, Joel S. Emer, and Daniel Sanchez. [n. d.]. Trapezoid: A Versatile Accelerator for Dense and Sparse Matrix Multiplications. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.

[56] Raphael Yuster and Uri Zwick. 2004. Detecting Short Directed Cycles using Rectangular Matrix Multiplication and Dynamic Programming.. In *SODA*, Vol. 4. 254–260.

[57] Xiaoyu Zhang, Zerun Li, Rui Liu, Xiaoming Chen, and Yinhe Han. 2024. GAS: General-Purpose In-Memory-Computing Accelerator for Sparse Matrix Multiplication. *IEEE Trans. Comput.* (2024).

[58] Yicong Zhang, Mingyu Wang, Yangzhan Mai, and Zhiyi Yu. 2023. TensorCache: Reconstructing Memory Architecture With SRAM-Based In-Cache Computing for Efficient Tensor Computations in GPGPUs. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2023).

[59] Yicong Zhang, Mingyu Wang, Wangguang Wang, Yangzhan Mai, Haiqiu Huang, and Zhiyi Yu. 2024. Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs Through In-Cache Atomic Operations. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 671–685.

[60] Zhekai Zhang, Hanrui Wang, Song Han, and William J Dally. 2020. Sparch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 261–274.

[61] Maohua Zhu, Tao Zhang, Zhenyu Gu, and Yuan Xie. 2019. Sparse Tensor Core: Algorithm and Hardware Co-design for Vector-Wise Sparse Neural Networks on Modern GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 359–371.